多线程 + 多TCP连接优化方案（屏幕共享独立通道）

一、优化背景

1.1 问题现象

前几轮优化前屏幕共享功能正常，但在后续优化过程中屏幕共享功能出现问题。

1.2 根本原因

原架构中客户端仅使用 **2 条 TCP 连接**：

- `audio_client`：音频专用连接
- `video_client`：视频专用连接（**屏幕共享复用此连接**）

屏幕共享和摄像头视频共用同一条 TCP 连接和同一个协议类型（`DEF_PACK_VIDEO_FRAME`），导致：

1. **滑动窗口阻塞**：视频数据量大时，屏幕共享数据被阻塞
2. **协议混淆**：屏幕共享帧和视频帧使用相同协议类型，无法区分
3. **线程竞争**：屏幕共享和视频共用同一个发送线程（`SendVideoWorker`）

1.3 优化方案

采用 **多线程优化结构 + 多条TCP连接优化**：

- 客户端：3 条独立 TCP 连接（音频、视频、屏幕共享）
- 客户端：3 个独立发送线程（`SendVideoWorker`、`SendScreenWorker` 等）
- 服务端：`UserInfo` 新增 `m_screenfd`，屏幕共享帧通过独立 fd 转发
- 协议：新增 `DEF_PACK_SCREEN_REGISTER` 和 `DEF_PACK_SCREEN_FRAME`

二、服务端代码改动（★重点★手动修改清单）

> **以下为服务端需要手动修改的代码，共 3 个文件 4 处改动**

★改动1：服务端协议定义文件 服务端代码/include/packdef.h

改动1.1：新增屏幕共享协议类型（在 DEF_PACK_UPDATE_EMAIL_RS 后面）

定位：文件第 80 行附近，DEF_PACK_UPDATE_EMAIL_RS 定义之后

新增代码：

```
//【多TCP连接优化-新增】屏幕共享注册和屏幕共享帧协议
//将屏幕共享从视频连接中独立出来，使用独立的TCP连接和协议
//防止视频帧和屏幕共享帧在同一条连接上互相阻塞（滑动窗口阻塞）
#define DEF_PACK_SCREEN_REGISTER  (DEF_PACK_BASE + 17)
#define DEF_PACK_SCREEN_FRAME     (DEF_PACK_BASE + 18)
```

改动1.2：UserInfo 结构体新增 m_screenfd 字段

定位：文件第 295 行附近，UserInfo 结构体定义

修改前：

```
typedef struct UserInfo
{
    UserInfo()
    {
        m_sock = 0;
        m_id = 0;
        m_roomid = 0;
        memset(m_userName , 0 , MAX_SIZE);
        m_videofd = 0;
        m_audiofd = 0;
    }
    int  m_sock;
    int  m_id;
    int m_roomid;
    char m_userName[MAX_SIZE];

    int m_videofd;
    int m_audiofd;

}UserInfo;
```

修改后：

```
typedef struct UserInfo
{
    UserInfo()
```

```
    {
        m_sock = 0;
        m_id = 0;
        m_roomid = 0;
        memset(m_userName , 0 , MAX_SIZE);
        m_videofd = 0;
        m_audiofd = 0;
        //【多TCP连接优化-新增】屏幕共享专用fd
        m_screenfd = 0;
    }
    int  m_sock;
    int  m_id;
    int m_roomid;
    char m_userName[MAX_SIZE];

    int m_videofd;
    int m_audiofd;
    //【多TCP连接优化-新增】屏幕共享专用fd
    int m_screenfd;

}UserInfo;
```

改动1.3：新增 STRU_SCREEN_REGISTER 结构体（在 STRU_UPDATE_EMAIL_RS 后面）

定位：文件第 370 行附近，STRU_UPDATE_EMAIL_RS 定义之后

新增代码：

```
//【多TCP连接优化-新增】屏幕共享注册
struct STRU_SCREEN_REGISTER
{
    STRU_SCREEN_REGISTER() : m_nType(DEF_PACK_SCREEN_REGISTER)
    {
        m_userid = 0;
    }
    PackType m_nType;//包类型
    int m_userid;//用户ID
};
```

★改动2：服务端业务逻辑头文件 服务端代码/include/clogic.h

定位：VideoRegister 函数声明之后，UpdateEmailRq 之前

新增代码：

```
    //【多TCP连接优化-新增】屏幕共享注册
    void ScreenRegister(int clientfd, char* szbuf, int nlen);
```

```
    //【多TCP连接优化-新增】屏幕共享帧处理
    void ScreenFrameRq(int clientfd, char* szbuf, int nlen);
```

★改动3：服务端业务逻辑实现文件 服务端代码/src/clogic.cpp

改动3.1：注册协议映射（在 setNetPackMap 函数中）

定位：文件开头 setNetPackMap 函数，DEF_PACK_UPDATE_EMAIL_RQ 映射之后

新增代码：

```
    //【多TCP连接优化-新增】注册屏幕共享协议映射
    NetPackMap(DEF_PACK_SCREEN_REGISTER)     =&CLogic::ScreenRegister;
    NetPackMap(DEF_PACK_SCREEN_FRAME)        =&CLogic::ScreenFrameRq;
```

改动3.2：新增 ScreenRegister 和 ScreenFrameRq 函数实现

定位：VideoRegister 函数之后，UpdateEmailRq 函数之前

新增代码：

```
//【多TCP连接优化-新增】屏幕共享注册
void CLogic::ScreenRegister(int clientfd, char *szbuf, int nlen)
{
    printf("clientfd:%d ScreenRegister\n", clientfd);
    //拆包
    STRU_SCREEN_REGISTER *rq = (STRU_SCREEN_REGISTER *)szbuf;
    int userid = rq->m_userid;
    //根據userid找到節點 更新fd
    if( m_mapIDToUserInfo.IsExist(userid) )
    {
        UserInfo * info = m_mapIDToUserInfo.find(userid);
        info->m_screenfd = clientfd;
    }
}

//【多TCP连接优化-新增】屏幕共享帧处理
void CLogic::ScreenFrameRq(int clientfd, char *szbuf, int nlen)
{
    printf("clientfd:%d ScreenFrameRq\n", clientfd);
    //拆包
    char * tmp = szbuf;
    //跳過type
    tmp += sizeof(int);
    //讀取userid
    int uid = *(int*) tmp;
```

```
    //跳過userid
    tmp += sizeof(int);
    //獲取roomid
    int roomid = *(int*) tmp;

    //看房間是否存在
    if( !m_mapIDToRoomid.IsExist( roomid )) return;
    list<int> lst = m_mapIDToRoomid.find(roomid);
    //獲取成員列表
    for( auto ite = lst.begin() ; ite != lst.end() ; ++ite)
    {
        //看是否在線，轉發
        int userid = *ite;
        //屏蔽掉自己 不轉發
        if( uid == userid ) continue;
        if( !m_mapIDToUserInfo.IsExist(userid)) continue;
        UserInfo * userinfo = m_mapIDToUserInfo.find(userid);
        //【多TCP连接优化】通过 m_screenfd 转发屏幕共享帧
        if( userinfo->m_screenfd != 0 )
            SendData( userinfo->m_screenfd , szbuf , nlen);
        else
            SendData( userinfo->m_sock , szbuf , nlen);
    }
}
```

三、服务端改动文件清单总结

序号	文件路径	改动类型	改动内容
1	服务端代码/include/packdef.h	新增协议	DEF_PACK_SCREEN_REGISTER、DEF_PACK_SCREEN_FRAME
2	服务端代码/include/packdef.h	修改结构体	UserInfo 新增 m_screenfd 字段
3	服务端代码/include/packdef.h	新增结构体	STRU_SCREEN_REGISTER
4	服务端代码/include/clogic.h	新增声明	ScreenRegister、ScreenFrameRq
5	服务端代码/src/clogic.cpp	新增映射	DEF_PACK_SCREEN_REGISTER、DEF_PACK_SCREEN_FRAME 映射

序号	文件路径	改动类型	改动内容
6	服务端代码/src/clogic.cpp	新增实现	ScreenRegister、ScreenFrameRq 函数

四、客户端代码改动（已自动完成）

4.1 客户端协议层 WeChat/netapi/net/packdef.h

- 新增 DEF_PACK_SCREEN_REGISTER 和 DEF_PACK_SCREEN_FRAME 协议
- 新增 STRU_SCREEN_REGISTER 结构体

4.2 客户端核心头文件 WeChat/ckernel.h

- client_type 枚举新增 screen_client = 2
- m_pAVClient 数组从 2 个扩展为 3 个
- 新增 SIG_SendScreen 信号
- 新增 slot_sendScreenFrame、slot_SendScreen、slot_dealScreenFrameRq 槽函数
- 新增 m_pSendScreenWorker 成员
- 新增 SendScreenWorker 类

4.3 客户端核心实现 WeChat/ckernel.cpp

- 构造函数：创建 3 条 TCP 连接，屏幕共享信号连接到独立槽函数
- 析构函数：回收 3 条 TCP 连接
- slot_dealLoginRS：登录成功后发送 STRU_SCREEN_REGISTER 注册包
- 新增 slot_sendScreenFrame：使用 DEF_PACK_SCREEN_FRAME 协议打包
- 新增 slot_SendScreen：通过 m_pAVClient[screen_client] 发送
- 新增 slot_dealScreenFrameRq：处理接收的屏幕共享帧

五、优化架构对比

5.1 优化前架构（2 条 TCP 连接）

```
客户端                                    服务端
┌──────────────┐                          ┌──────────────┐
```

```
┌ audio_client├──────────────────────┤ m_audiofd   │
├──────────────┤                      ├─────────────┤
┌ video_client├──────────────────────┤ m_videofd   │
│  (视频+屏幕) │                      │  (视频+屏幕) │
└──────────────┘                      └─────────────┘

       ↑
       └── 滑动窗口阻塞！视频和屏幕共享互相竞争
```

5.2 优化后架构（3 条 TCP 连接）

```
客户端                                服务端

┌──────────────┐                      ┌─────────────┐
┌ audio_client├──────────────────────┤ m_audiofd   │
├──────────────┤                      ├─────────────┤
┌ video_client├──────────────────────┤ m_videofd   │
├──────────────┤                      ├─────────────┤
┌screen_client├──────────────────────┤ m_screenfd  │
│  (屏幕专用)  │                      │  (屏幕专用)  │
└──────────────┘                      └─────────────┘

       ↑
       └── 3 条连接互相独立，无滑动窗口阻塞
```

5.3 多线程优化结构

```
主线程
  │
  ├── SIG_SendVideo ──→ SendVideoWorker 线程 ──→ video_client TCP
  │
  └── SIG_SendScreen ─→ SendScreenWorker 线─→ screen_client TCP
```

六、验证要点

1. **服务端编译**：确认 3 个文件 6 处改动已应用，编译通过
2. **客户端编译**：确认客户端代码已自动修改完成，编译通过
3. **功能测试**：
   - 登录后检查服务端日志是否有 `ScreenRegister` 输出
   - 开启屏幕共享后检查服务端日志是否有 `ScreenFrameRq` 输出
   - 验证视频和屏幕共享同时开启时互不卡顿
4. **滑动窗口阻塞验证**：同时开启摄像头和屏幕共享，观察是否流畅

七、多线程安全补充：QImage 隐式共享的线程安全陷阱（2026-07-29 补充）

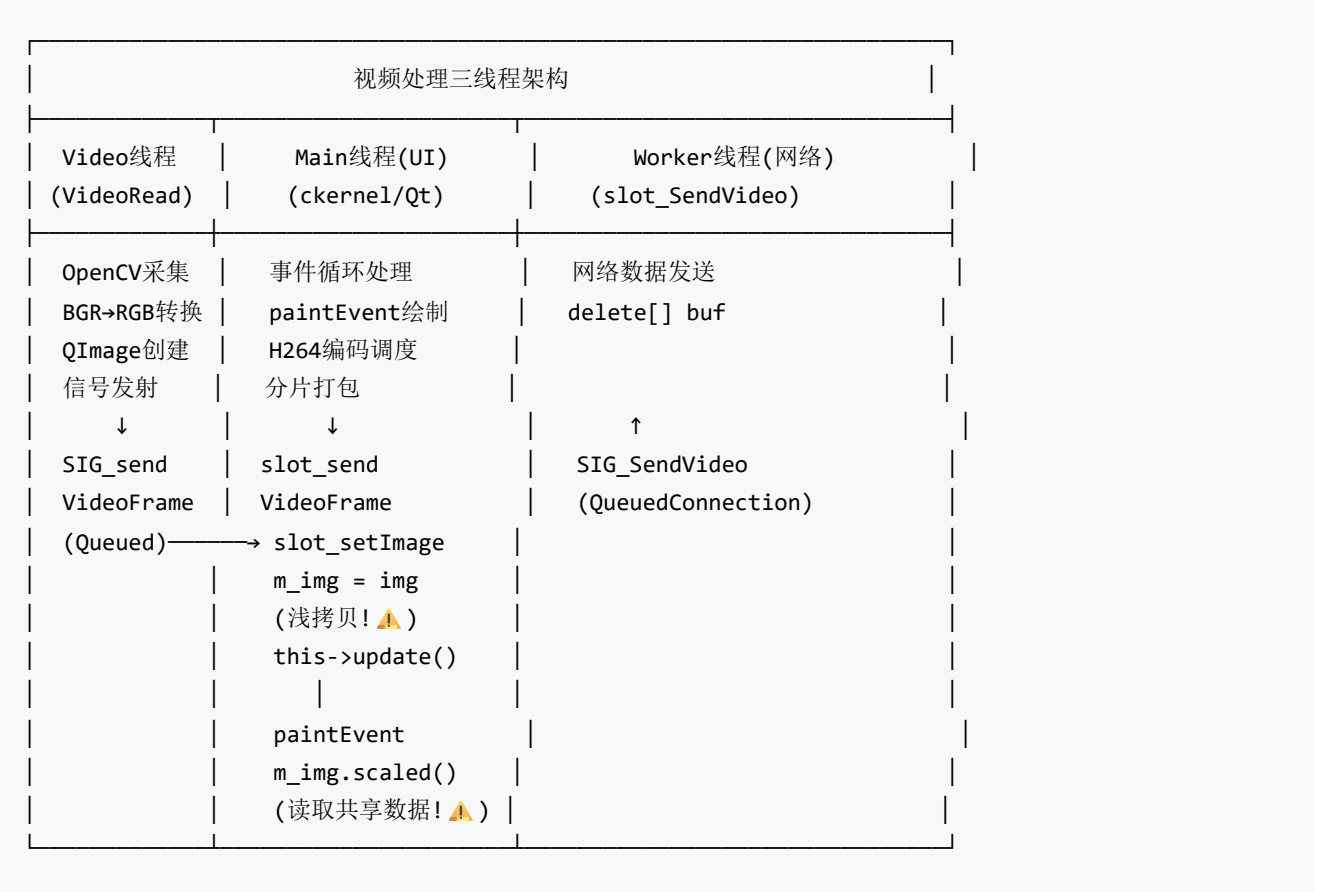
> 本节为 H264 视频编码崩溃修复过程中发现的多线程安全问题。在多线程架构下，Qt 隐式共享类型的浅拷贝会导致严重的竞态条件。详细内容参考 H264视频崩溃修复报告.md。

7.1 问题背景

本文档前述的多线程优化方案（多 TCP 连接 + 多发送线程）解决了网络层面的滑动窗口阻塞和线程竞争问题。但在视频处理链路中，还存在一个隐蔽的**线程安全陷阱**：Qt 隐式共享（Implicit Sharing）机制在多线程环境下的非原子引用计数操作。

启用 H264 编码后，视频处理链路涉及三个线程的并发操作，QImage 浅拷贝导致客户端在第一帧编码发送完成后立即崩溃。

7.2 三线程架构中的 QImage 竞态条件



7.3 竞态条件分析

7.3.1 QImage 隐式共享机制

Qt 的 QImage 使用**隐式共享**（Copy-On-Write）机制：

```
QImage A(frame.data, 640, 480, RGB888);  // A 持有数据
QImage B = A;                             // 浅拷贝：A 和 B 共享同一块像素数据
                                          //         引用计数 refcount = 2
```

m_img = img 只是引用计数 +1，并不复制底层数据。

7.3.2 竞态窗口

线程	操作	对共享数据的影响
Video线程	持续创建新 QImage 并通过 SIG_sendVideoFrame 发送	新 QImage 的创建和旧 QImage 的销毁会改变共享数据的引用计数
Main线程	paintEvent 中 m_img.scaled() 读取像素数据	scaled() 是只读操作，但需要通过 d-pointer 访问共享数据
Worker线程	delete[] buf 执行堆操作	堆的 free 操作可能改变堆内存布局，加速共享数据块被回收

竞态窗口：当 Main 线程的 m_img.scaled() 正在通过 d-pointer 遍历像素数据时，如果 Video 线程的 QImage 析构将 refcount 减至 0 并释放了像素数据块，Main 线程将读取已释放内存 → 访问违规崩溃。

7.4 深拷贝最佳实践

7.4.1 核心修复：slot_setImage 深拷贝

```
// ❌ 错误：跨线程传递 QImage 使用浅拷贝
void UserShow::slot_setImage(QImage &img)
{
    m_img = img;  // 浅拷贝，多线程下不安全
    this->update();
}

// ✅ 正确：跨线程传递 QImage 使用深拷贝
void UserShow::slot_setImage(QImage &img)
{
    m_img = img.copy();  // 深拷贝，独立像素数据
    this->update();
}
```

7.4.2 Lambda 捕获 QImage 深拷贝

```
// ❌ 错误：Lambda 捕获 QImage 使用浅拷贝
QTimer::singleShot(0, this, [this, img]() {
    encodeAndSendVideoFrame(img);  // img 可能已被释放
});

// ✅ 正确：Lambda 捕获 QImage 使用深拷贝
QImage imgCopy = img.copy();  // 深拷贝，独立像素数据
QTimer::singleShot(0, this, [this, imgCopy]() {
    encodeAndSendVideoFrame(imgCopy);
});
```

7.5 隐式共享类型清单

以下 Qt 类型使用隐式共享，跨线程传递时均需注意深拷贝：

类型	用途	跨线程风险
QImage	视频帧像素数据	高（像素数据可能被释放）
QPixmap	显示图像	高（同 QImage）
QByteArray	网络数据缓冲区	中（数据可能被覆盖）
QString	字符串	低（字符串通常不可变）
QList / QVector	列表数据	中（数据可能被修改）
QMap	映射数据	中（数据可能被修改）
QPen / QBrush / QFont	绘制属性	低（通常不可变）

7.6 多线程安全检查清单

在多线程架构下使用 Qt 隐式共享类型时，建议按以下清单检查：

- [ ] 跨线程信号槽传递 QImage/QPixmap 是否使用了 `copy()` 深拷贝
- [ ] Lambda 捕获 QImage/QByteArray 是否使用了深拷贝
- [ ] 成员变量赋值 QImage 是否使用了 `copy()` 深拷贝
- [ ] 网络发送线程的 `delete[]` 操作是否与 UI 线程的像素读取存在竞态
- [ ] 视频采集线程的 QImage 生命周期是否会影响 UI 线程的共享数据

7.7 与多 TCP 连接优化的关系

本文档前述的多 TCP 连接优化解决了**网络层面**的线程竞争（滑动窗口阻塞），而 QImage 深拷贝修复解决了**数据层面**的线程竞争（引用计数竞态）。两者共同构成了视频会议系统的多线程安全基础：

优化层面	问题	修复方案
网络层	滑动窗口阻塞、协议混淆	多 TCP 连接 + 多发送线程
数据层	QImage 引用计数竞态	`img.copy()` 深拷贝
编码层	FFmpeg 帧缓冲区共享	`av_frame_get_buffer` + `av_frame_make_writable`